

notebook

February 6, 2026

1 EJERCICIO FINAL PYSPARK (DOCKER + KAFKA + PYS-PARK)

```
[9]: from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

1.1 Creamos la sesión de spark

```
[10]: spark = (
    SparkSession.builder
        .appName("ProyectoIoT")
        .getOrCreate()
)
spark
```

```
[10]: <pyspark.sql.session.SparkSession at 0x7fc493e94cd0>
```

spark.readStream Indica que vamos a leer datos en modo streaming, es decir, datos que llegan continuamente.

.format("kafka") Le decimos a Spark que la fuente de datos de streaming será Kafka.

.option("kafka.bootstrap.servers", "kafka:9092") Especifica la dirección del clúster Kafka al que conectarse. En este caso "kafka" es el nombre del contenedor Docker y 9092 es el puerto del broker.

.option("subscribe", "sensores") Indica el topic de Kafka del que queremos leer mensajes → "sensores".

.option("startingOffsets", "latest") Define desde qué punto empezar a leer:

"latest" → solo recibe mensajes nuevos, enviados después de iniciar el streaming.

"earliest" sería para leer todos los mensajes antiguos también.

.load() Ejecuta la configuración y crea un DataFrame de streaming, donde cada fila representa un mensaje recibido desde Kafka.

```
[11]: raw_df = (
    spark.readStream
      .format("kafka")
      .option("kafka.bootstrap.servers", "kafka:9092") # <-- nombre del
      ↪ contenedor
      .option("subscribe", "sensores")
      .option("startingOffsets", "latest")
      .load()
  )
```

Este bloque transforma los mensajes de Kafka en un formato que Spark puede procesar:

Define la estructura de los datos que esperamos recibir (sensor, valor, temperatura, humedad, estado, timestamp y UUID).

Convierte el JSON recibido en columnas individuales para poder trabajar con cada campo.

Crea una columna de tiempo (event_time) a partir del timestamp original para poder usarlo en ventanas y agregaciones temporales.

```
[12]: schema = StructType([
    StructField("sensor_id", StringType()),
    StructField("value", DoubleType()),
    StructField("temperature", DoubleType()),
    StructField("humidity", DoubleType()),
    StructField("status", StringType()),
    StructField("timestamp", DoubleType()),
    StructField("uuid", StringType())
])

df = raw_df.select(
    from_json(col("value").cast("string"), schema).alias("data")
).select("data.*")

df = df.withColumn("event_time", col("timestamp").cast("timestamp"))
```

Este bloque realiza agregaciones por ventana de tiempo sobre el DataFrame de streaming:

Define un watermark de 1 minuto para indicar a Spark hasta qué punto los datos antiguos se pueden considerar válidos. Esto ayuda a manejar retrasos y datos tardíos.

Agrupar los datos cada 30 segundos por sensor (sensor_id).

Calcula estadísticas dentro de cada ventana: promedio de valor, temperatura y humedad, y cuenta el número de eventos recibidos.

El resultado es un DataFrame de streaming con resúmenes temporales por sensor listo para análisis o escritura.

```
[13]: agg_df = (
    df
```

```

.withWatermark("event_time", "1 minute")  # ← IMPORTANTE
.groupBy(
    window(col("event_time"), "30 seconds"),
    col("sensor_id")
)
.agg(
    avg("value").alias("avg_value"),
    avg("temperature").alias("avg_temp"),
    avg("humidity").alias("avg_humidity"),
    count("*").alias("num_events")
)
)

```

Este bloque escribe los resultados del streaming en archivos Parquet de manera continua:

Define la carpeta de salida donde se guardarán los archivos Parquet (resultados/).

Usa un checkpoint (chk/) para que Spark recuerde qué datos ya procesó y pueda reiniciar de forma segura en caso de fallo.

Modo append: los nuevos datos se agregan continuamente a los archivos existentes.

Inicia el streaming, haciendo que las agregaciones se escriban en tiempo real mientras llegan nuevos datos.

El resultado es un conjunto de archivos Parquet que refleja las métricas agregadas por ventana y por sensor.

```

[14]: parquet_query = (
    agg_df
    .writeStream
    .format("parquet")
    .option("path", "resultados/")
    .option("checkpointLocation", "chk/")
    .outputMode("append")
    .start()
)

```

A partir de aquí, te toca a ti, sigue las cuestiones planteadas en el Readme y completa el notebook. Después guardatelo con los outputs de las celdas y súbelo al repo. Mucha suerte que ya lo teneis ;)

```

[15]: df_agg = spark.read.parquet("resultados/")
df_agg.show(5)

```

```

+-----+-----+-----+-----+-----+
+-----+-----+
|           window|sensor_id|           avg_value|           avg_temp|
avg_humidity|num_events|
+-----+-----+-----+-----+-----+
+-----+-----+

```

```
|{2026-02-06 22:25...|      S4|      49.86|      60.9|
50.84|      1|
|{2026-02-06 22:26...|      S3|      31.715|      56.5425|
51.88833333333334|      12|
|{2026-02-06 22:26...|      S4|26.433333333333337| 65.97666666666667|
49.79666666666666|      12|
|{2026-02-06 22:26...|      S1|26.420000000000005|49.096000000000004|
62.39|      5|
|{2026-02-06 22:26...|      S4| 35.83111111111111|
58.42777777777778|57.623333333333335|      9|
+-----+-----+-----+-----+-----+
-----+-----+
only showing top 5 rows
```

1.1.1 Ejercicio 1

```
[16]: df_agg.printSchema()
# Contar total de registros
print(f"Total filas: {df_agg.count()}")
# Contar sensores únicos
num_sensores = df_agg.select('sensor_id').distinct().count()
print(f"Sensores distintos: {num_sensores}")
```

```
root
|-- window: struct (nullable = false)
|   |-- start: timestamp (nullable = true)
|   |-- end: timestamp (nullable = true)
|-- sensor_id: string (nullable = true)
|-- avg_value: double (nullable = true)
|-- avg_temp: double (nullable = true)
|-- avg_humidity: double (nullable = true)
|-- num_events: long (nullable = false)
```

```
Total filas: 38
Sensores distintos: 4
```

1.1.2 Ejercicio 2

```
[17]: from pyspark.sql.functions import col

df_trans = df_agg.withColumn(
    "temp_humidity_diff", col("avg_temp") - col("avg_humidity")
).withColumn(
    "value_per_event", col("avg_value") / col("num_events")
)
df_trans.select("temp_humidity_diff").summary().show()
```

```

+-----+-----+
|summary| temp_humidity_diff|
+-----+-----+
|  count|                38|
|   mean|  0.775325918964076|
| stddev| 15.195389156705957|
|   min|                -36.84|
|  25%|-5.1610000000000085|
|  50%|  0.8044444444444423|
|  75%|  6.349166666666655|
|   max|  60.50000000000001|
+-----+-----+

```

1.1.3 Ejercicio 3

```

[18]: df_filtered = df_agg.filter(
      (col("avg_humidity") > 50) &
      (col("num_events") >= 25) &
      col("sensor_id").isin(["S1", "S3"])
    )
      # Contar resultados
      print(f"Registros filtrados: {df_filtered.count()}")

```

Registros filtrados: 0

1.1.4 Ejercicio 4

```

[19]: import pyspark.sql.functions as F

      df_sensor_stats = df_agg.groupBy("sensor_id").agg(
      F.avg("avg_value").alias("media_valor"),
      F.max("avg_temp").alias("temp_maxima"),
      F.count("*").alias("num_ventanas")
      )
      df_sensor_stats.show()

```

```

+-----+-----+-----+-----+
|sensor_id|      media_valor|      temp_maxima|num_ventanas|
+-----+-----+-----+-----+
|      S4|33.746789682539685|65.97666666666667|          10|
|      S3| 28.35534656084656|          73.93|           9|
|      S1| 24.980611111111111|          85.65|           9|
|      S2|31.030228968253972|64.48666666666666|          10|
+-----+-----+-----+-----+

```

1.1.5 Ejercicio 5

```
[20]: from pyspark.sql.window import Window
from pyspark.sql.functions import col, rank, desc

window_spec = Window.partitionBy("sensor_id").orderBy(col("avg_value").desc())
df_ranked = df_agg.withColumn("rank", rank().over(window_spec))
df_top = df_ranked.filter(col("rank") == 1)
df_top.show()
```

```
+-----+-----+-----+-----+-----+-----+
-----+-----+
|           window|sensor_id|           avg_value|avg_temp|
avg_humidity|num_events|rank|
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+
|{2026-02-06 22:28...|      S1|37.163749999999999|66.64125|           54.69125|
8|      1|
|{2026-02-06 22:26...|      S2|           41.086|   34.068|63.612000000000001|
5|      1|
|{2026-02-06 22:26...|      S3|           31.715|  56.5425|51.888333333333334|
12|      1|
|{2026-02-06 22:25...|      S4|           49.86|   60.9|           50.84|
1|      1|
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+
```

1.1.6 Ejercicio 6

```
[21]: sensor_info = spark.createDataFrame([
("S1", "Exterior", "Temperatura"),
("S2", "Interior", "Humedad"),
("S3", "Exterior", "Multisensor"),
("S4", "Interior", "Temperatura")
], ["sensor_id", "ubicacion", "categoria"])

df_joined = df_agg.join(sensor_info, "sensor_id", "left")

df_joined.show(5)
```

```
+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
|sensor_id|           window|           avg_value|           avg_temp|
avg_humidity|num_events|ubicacion| categoria|
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
|      S4|{2026-02-06 22:25...|           49.86|           60.9|
```

```

50.84|          1| Interior|Temperatura|
|          S4|{2026-02-06 22:26...|26.433333333333337| 65.97666666666667|
49.796666666666666|          12| Interior|Temperatura|
|          S4|{2026-02-06 22:26...| 35.831111111111111|
58.42777777777778|57.623333333333335|          9| Interior|Temperatura|
|          S4|{2026-02-06 22:28...| 32.90428571428571|54.074285714285715|
56.97714285714285|          7| Interior|Temperatura|
|          S3|{2026-02-06 22:26...|          31.715|          56.5425|
51.88833333333334|          12| Exterior|Multisensor|
+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
only showing top 5 rows

```

1.1.7 Ejercicio 7

```

[22]: from pyspark.sql.functions import date_trunc, col
import pyspark.sql.functions as F

df_temporal = df_agg.withColumn(
    "minuto",
    date_trunc("minute", col("window.start"))
)

df_temporal.groupBy("minuto").agg(
    F.count("*").alias("ventanas"),
    F.avg("avg_humidity").alias("humedad_media")
).orderBy("minuto").show()

```

```

+-----+-----+-----+
|          minuto|ventanas|    humedad_media|
+-----+-----+-----+
|2026-02-06 22:25:00|      2|          55.715|
|2026-02-06 22:26:00|      8|51.776604166666665|
|2026-02-06 22:27:00|      8| 56.32500793650794|
|2026-02-06 22:28:00|      8| 54.62367063492063|
|2026-02-06 22:29:00|      8| 53.05247916666667|
|2026-02-06 22:30:00|      4| 57.62030555555556|
+-----+-----+-----+

```

1.1.8 Ejercicio 8

```

[23]: from pyspark.sql.functions import spark_partition_id
# Repartir en 4 particiones según el ID del sensor
df_repartitioned = df_agg.repartition(4, "sensor_id")
df_repartitioned = df_repartitioned.withColumn("partition_id",
    ↪spark_partition_id())

```

```
df_repartitioned.groupBy("partition_id").count().show()
```

```
+-----+-----+
|partition_id|count|
+-----+-----+
|          0|    9|
|          1|   20|
|          3|    9|
+-----+-----+
```

1.1.9 Ejercicio 9

```
[24]: # 1. Obtener media y desviación estándar globales
stats = df_agg.select(
    F.avg("avg_temp").alias("mean_temp"),
    F.stddev("avg_temp").alias("stddev_temp")
).collect()[0]

# 2. Definir criterios de anomalía
df_anomalies = df_agg.withColumn("es_anomalia",
    (col("avg_temp") > stats["mean_temp"] + 2*stats["stddev_temp"]) |
    (col("avg_temp") < stats["mean_temp"] - 2*stats["stddev_temp"]) |
    (col("avg_humidity") > 85) | (col("avg_humidity") < 25) |
    (col("num_events") < 15)
).withColumn("es_anomalia", F.when(col("es_anomalia"), True).otherwise(False))

print(f"Anomalías totales: {df_anomalies.filter(col('es_anomalia') == True).
    ↪count()}")
```

Anomalías totales: 38

1.1.10 Ejercicio 10

```
[25]: df_score = df_agg.withColumn("puntuacion",
    col("avg_value") * 0.4 + col("avg_temp") * 0.3 +
    col("avg_humidity") * 0.2 + col("num_events") * 0.1
)

df_score.groupBy("sensor_id").agg(
    F.avg("puntuacion").alias("puntuacion_promedio")
).orderBy(col("puntuacion_promedio").desc()).show()
```

```
+-----+-----+
|sensor_id|puntuacion_promedio|
+-----+-----+
|      S4| 42.326520634920634|
|      S2| 39.3784065079365|
```


	S1	39.25485092592592
	S3	38.975551851851854
+-----+-----+-----+		

1.1.11 Ejercicio 11

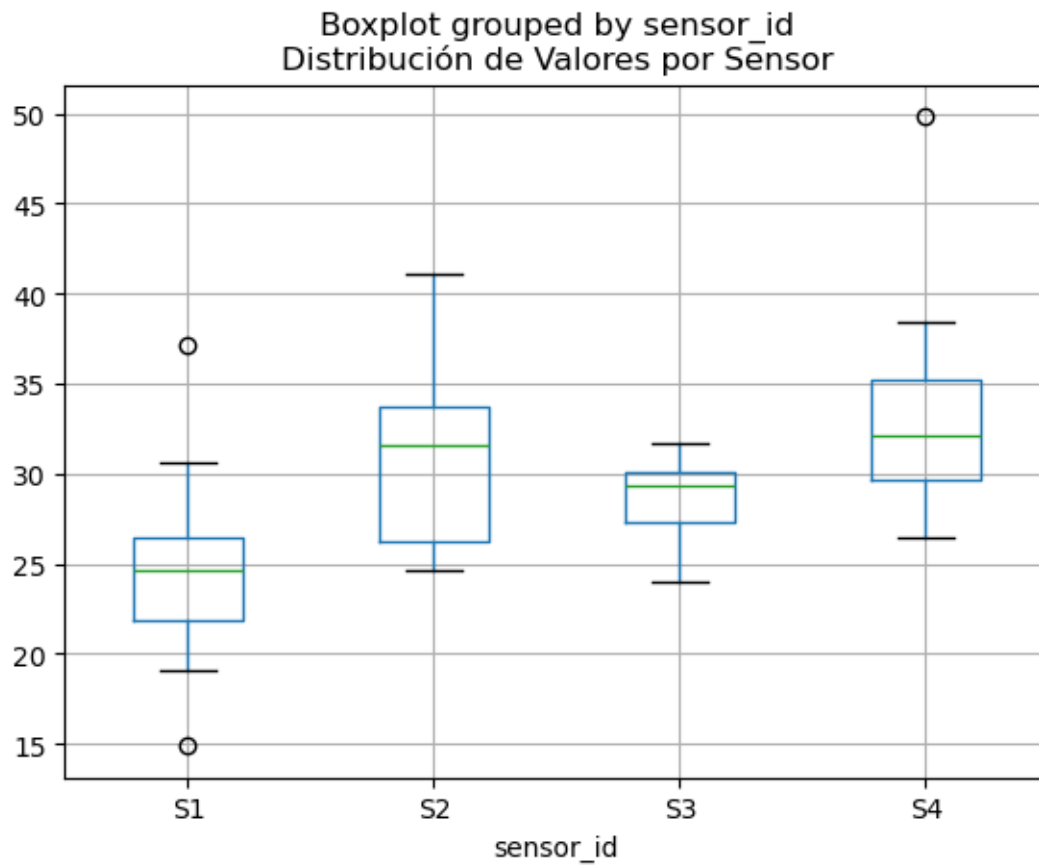
```
[27]: import matplotlib.pyplot as plt

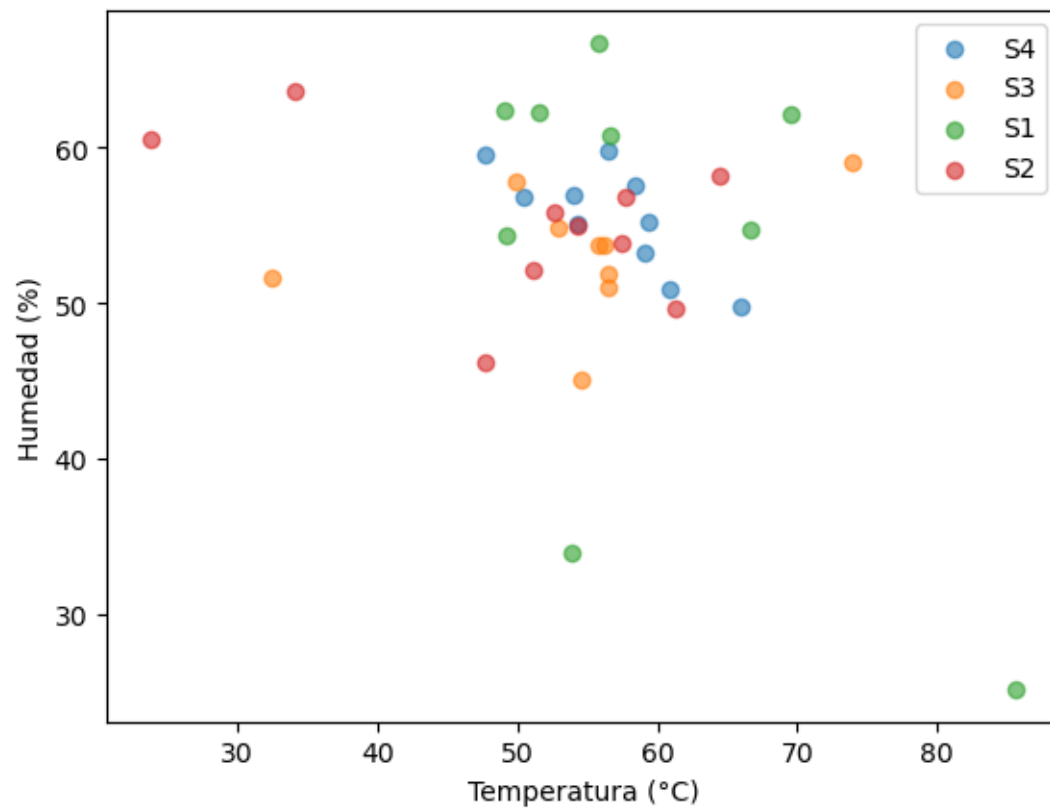
df_pandas = df_agg.toPandas()

df_pandas.boxplot(column='avg_value', by='sensor_id')
plt.title('Distribución de Valores por Sensor')
plt.show()

for sensor in df_pandas['sensor_id'].unique():
    data = df_pandas[df_pandas['sensor_id'] == sensor]
    plt.scatter(data['avg_temp'], data['avg_humidity'], label=sensor, alpha=0.6)

plt.xlabel('Temperatura (°C)')
plt.ylabel('Humedad (%)')
plt.legend()
plt.show()
```





[]: